Brewing the Soundness of BEer

Denotational Semantics, Set Theory and B

Vincent Trélat

Université de Lorraine, CNRS, Inria, LORIA, Nancy, France









1. Introduction
2. Encoding B POs in SMT-LIB using HO
3. Results
4. Denotational semantics of B
5. Soundness of the encoding

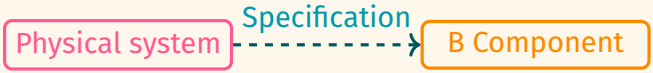
Introduction

B method

Physical system

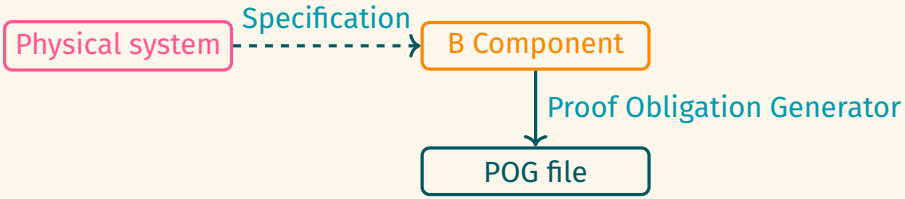
Introduction

B method



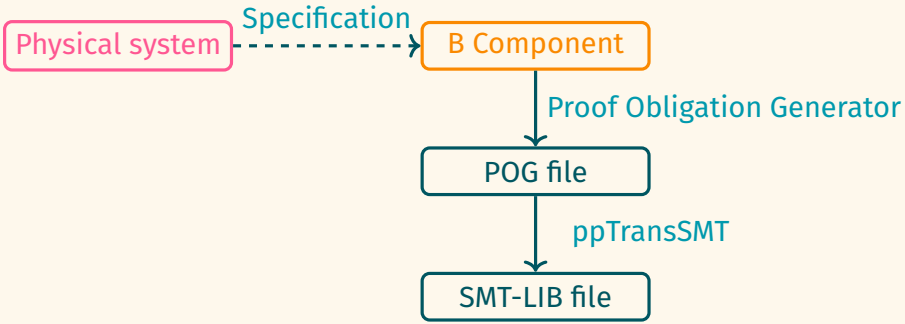
Introduction

B method



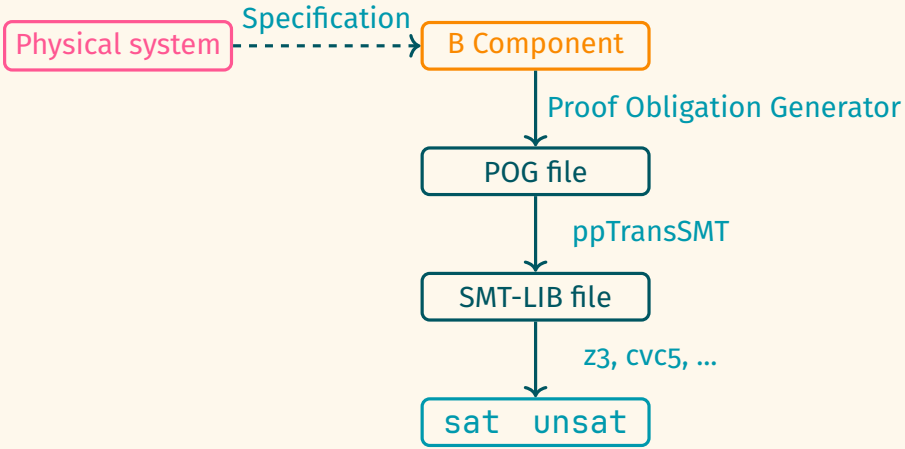
Introduction

B method



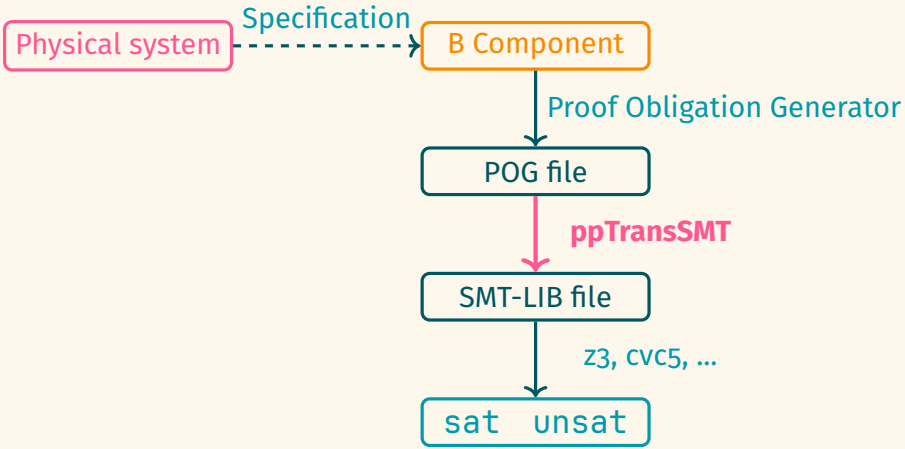
Introduction

B method



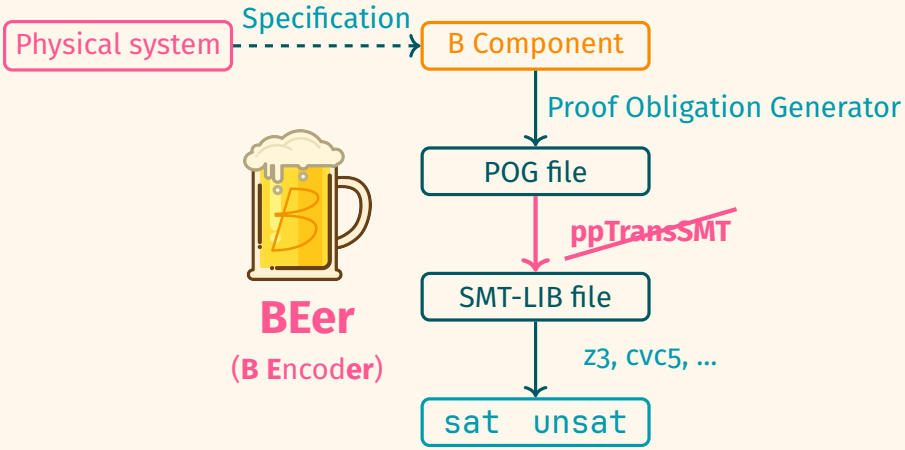
Introduction

B method



Introduction

B method



Introduction

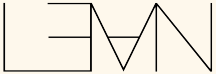
Because Germany taught me to lean on beer,

Introduction

Because Germany taught me to lean on beer,

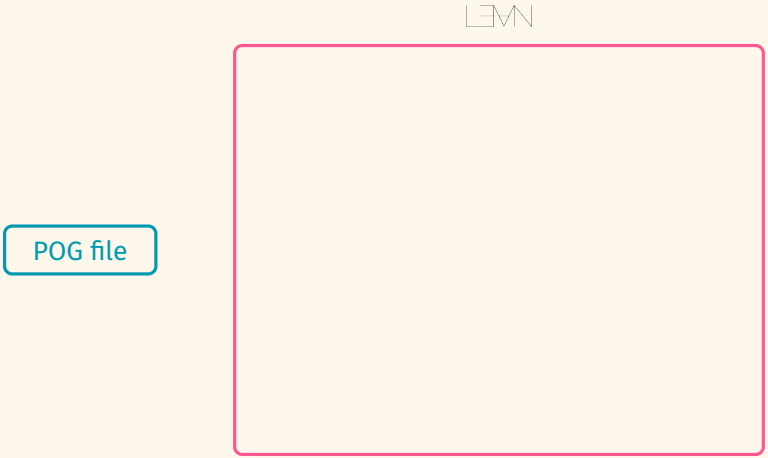


is written in

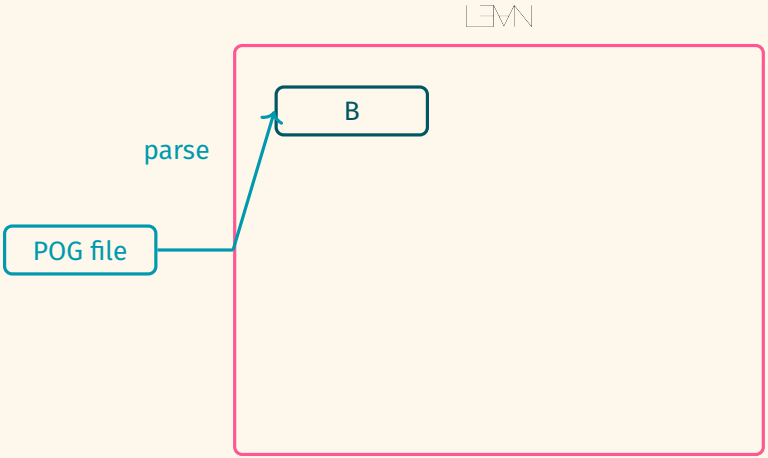


Architecture of 

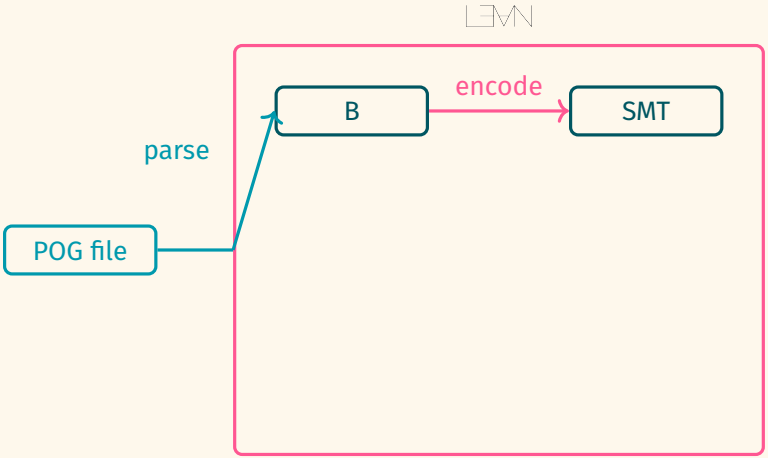
Architecture of 🍺



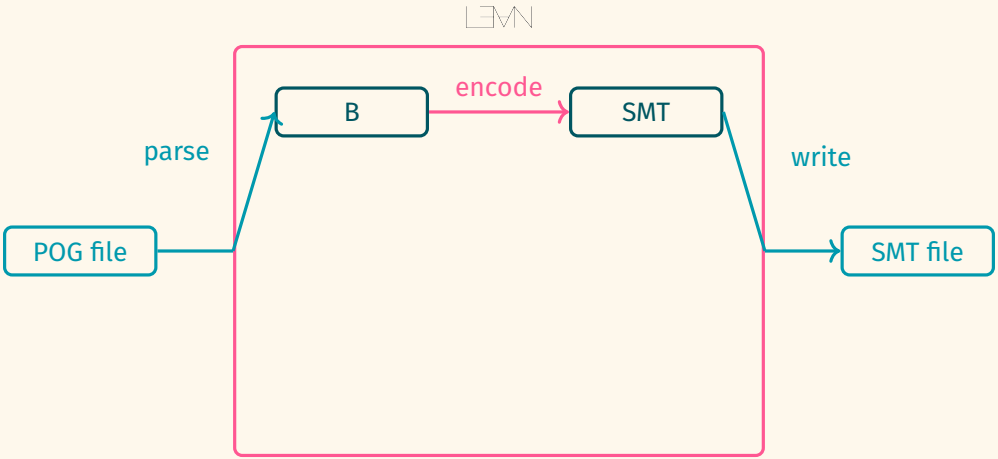
Architecture of



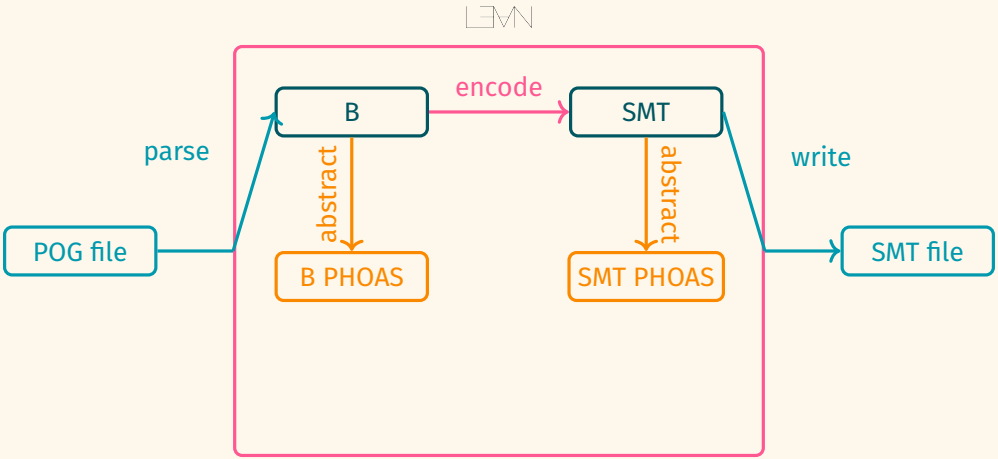
Architecture of 🍺



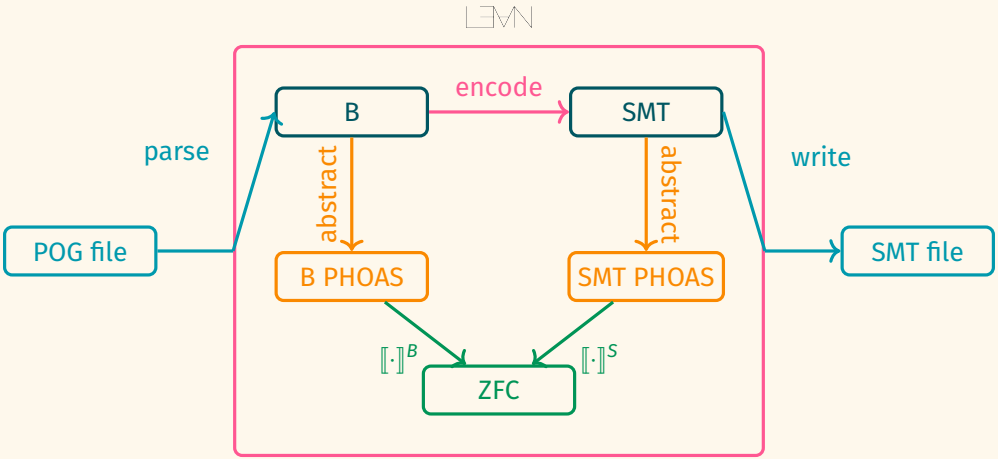
Architecture of 🍺



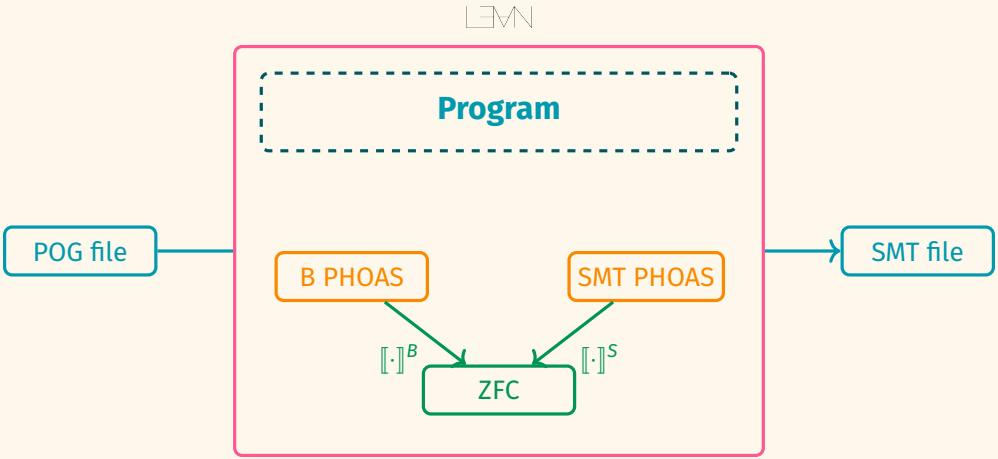
Architecture of



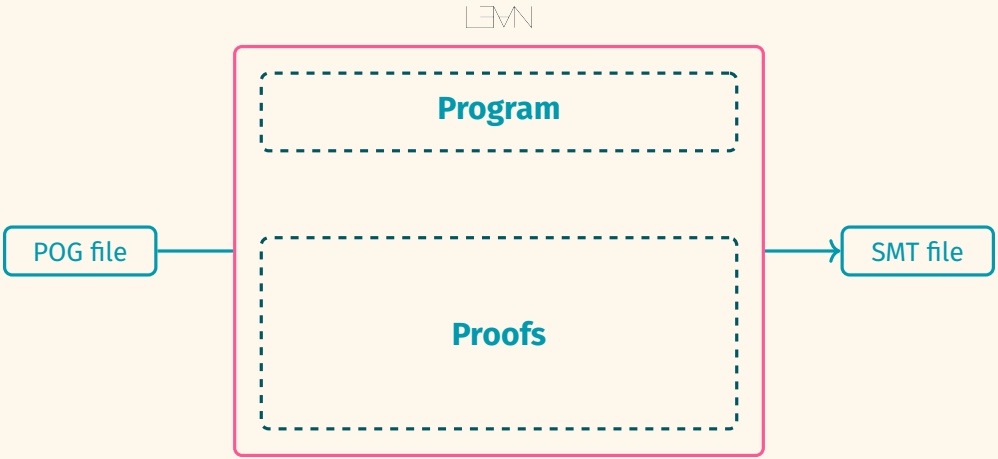
Architecture of



Architecture of 🍺



Architecture of 🍺



1 Introduction

2 Encoding B POs in SMT-LIB using HO
- Overview of the encoding

3 Results

4 Denotational semantics of B

5 Soundness of the encoding

Overview of the encoding

(former) first-order encoding

(new) higher-order encoding 🍺

Overview of the encoding

(former) first-order encoding

- FOL

(new) higher-order encoding 🍺

- HOL

Overview of the encoding

(former) first-order encoding

- FOL
- Specification of sets via $\in$, P and C

(new) higher-order encoding 🍺

- HOL
- Definition of sets via characteristic predicates

Overview of the encoding

```
SETS
  S = {e1, e2, e3}
```

Overview of the encoding

```
SETS
  S = {e1, e2, e3}
```

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-fun S () (P Int))
(declare-fun e1 () Int)
(declare-fun e2 () Int)
(declare-fun e3 () Int)
(assert (distinct e1 e2 e3))
(declare-fun ∈₀ ((Int) (P Int)) Bool)

(assert (forall ((x Int)) (=
  (∈₀ x S)
  (or (= x e1) (= x e2) (= x e3)))))
```

Overview of the encoding

```
SETS
  S = {e1, e2, e3}
```

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-fun S () (P Int))
(declare-fun e1 () Int)
(declare-fun e2 () Int)
(declare-fun e3 () Int)
(assert (distinct e1 e2 e3))
(declare-fun ∈₀ ((Int) (P Int)) Bool)

(assert (forall ((x Int)) (=
  (∈₀ x S)
  (or (= x e1) (= x e2) (= x e3)))))
```

(new) higher-order encoding

```
(declare-const e1 Int)
(declare-const e2 Int)
(declare-const e3 Int)
(assert (distinct e1 e2 e3))
(define-const S (→ Int Bool)
  (lambda ((x Int))
    (or (= x e1) (= x e2) (= x e3))))
```

Overview of the encoding

(former) first-order encoding

- FOL
- Specification of sets via $\in$, P and C

(new) higher-order encoding 🍺

- HOL
- Definition of sets via characteristic predicates

Overview of the encoding

(former) first-order encoding

- FOL
- Specification of sets via $\in$, `P` and `C`
- Only expressions like $x \in S$ are encoded

(new) higher-order encoding 🍺

- HOL
- Definition of sets via characteristic predicates
- Sets alone make sense; $x \in S$ is true by definition

Overview of the encoding

(former) first-order encoding

- FOL
- Specification of sets via $\in$, P and C
- Only expressions like $x \in S$ are encoded
- Functions are functional relations

(new) higher-order encoding 

- HOL
- Definition of sets via characteristic predicates
- Sets alone make sense; $x \in S$ is true by definition
- Functions are (sometimes) functions

Suppose we have a function $f \in A \nrightarrow B$.

Suppose we have a function $f \in A \nrightarrow B$.

(former) first-order encoding

f is a relation between A and B:

$$f \subseteq A \times B$$

f is functional:

$$\forall x\, y\, z,\ x \mapsto y \in f \ \wedge\ x \mapsto z \in f \Rightarrow y = z$$

Suppose we have a function $f \in A \nrightarrow B$.

(former) first-order encoding

f is a relation between A and B:

$$f \subseteq A \times B$$

f is functional:

$$\forall x\, y\, z,\ x \mapsto y \in f \ \wedge\ x \mapsto z \in f \Rightarrow y = z$$

(new) higher-order encoding 🍺

f is a total function from A to $B \cup \{\star\}$:

$$f \in (B \cup \{\star\})^A$$

Suppose we have a function $f \in A \nrightarrow B$.

(former) first-order encoding

f is a relation between A and B:

$$f \subseteq A \times B$$

f is functional:

$$\forall x\, y\, z,\ x \mapsto y \in f \ \wedge\ x \mapsto z \in f \Rightarrow y = z$$

(new) higher-order encoding 🍺

f is a total function from A to $B \cup \{\star\}$:

$$f \in (B \cup \{\star\})^A$$

```
(declare-datatype Option
  (par (T) ((some (the T)) (none))))
```

Suppose we have a function $f \in A \nrightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

Suppose we have a function $f \in A \nrightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-const f (P (C τ_A τ_B)))
(declare-fun
  ∈₀ (τ_A τ_B (P (C τ_A τ_B))) Bool)
(assert
  (forall ((x τ_A) (y τ_B) (z τ_B))
    (⇒ (and (∈₀ x y f) (∈₀ x z f))
       (= y z))))
```

Suppose we have a function $f \in A \nrightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-const f (P (C τ_A τ_B)))
(declare-fun
  ∈₀ (τ_A τ_B (P (C τ_A τ_B))) Bool)
(assert
  (forall ((x τ_A) (y τ_B) (z τ_B))
    (⇒ (and (∈₀ x y f) (∈₀ x z f))
       (= y z))))
```

(new) higher-order encoding

```
(declare-const f (→ τ_A (Option τ_B)))
```

Suppose we have a function $f \in A \nrightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-const f (P (C τ_A τ_B)))
(declare-fun
  ∈₀ (τ_A τ_B (P (C τ_A τ_B))) Bool)
(assert
  (forall ((x τ_A) (y τ_B) (z τ_B))
    (⇒ (and (∈₀ x y f) (∈₀ x z f))
       (= y z))))
```

(new) higher-order encoding

```
(declare-const f (→ τ_A (Option τ_B)))
```

+ specification that **dom** $f \subseteq A$ and **ran** $f \subseteq B$

1 Introduction

2 Encoding B POs in SMT-LIB using HO

3 Results

4 Denotational semantics of B

5 Soundness of the encoding

Results

In the current state of  we have:

ppTrans \ 	unsat	sat	unknown	Total
unsat	14,831	0	**1,062**	15,893
sat	0	0	0	0
unknown	**272**	0	780	1,052
Total	15,103	0	1,842	**16,945**

Benchmark specs:

- **681,285** POs in total
- Apple M2 (10 CPU cores, 24 GB RAM)
- cvc5 with incremental mode, MBQI enabled and 3s timeout per query

Results

In the current state of  we have:

ppTrans \ 	unsat	sat	unknown	Total
unsat	14,831	0	**1,062**	15,893
sat	0	0	0	0
unknown	**272**	0	780	1,052
Total	15,103	0	1,842	**16,945**

Benchmark specs:

- **681,285** POs in total
- Apple M2 (10 CPU cores, 24 GB RAM)
- cvc5 with incremental mode, MBQI enabled and 3s timeout per query

Conclusion: not that good on “simple” POs but performs well on HO POs!

1 Introduction

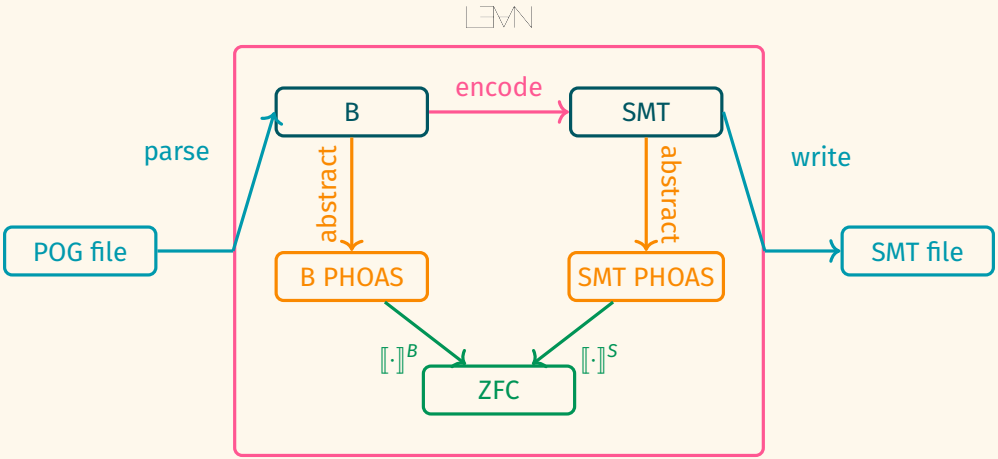
2 Encoding B POs in SMT-LIB using HO

3 Results

4 Denotational semantics of B
- Definitions
- Reasoning about B terms

5 Soundness of the encoding

Architecture of



Denotational semantics of B

We define a denotation function for abstract B terms:

$$[\![\cdot]\!]^{\mathrm{B}} \quad : \quad \mathtt{Term}\,\mathcal{V} \to \mathcal{V}$$

Denotational semantics of B

We define a denotation function for abstract B terms:

$$[\![\cdot]\!]^{\mathrm{B}} \quad : \quad \mathtt{Term}\ \mathtt{ZFSet} \to \mathtt{ZFSet}$$

Denotational semantics of B

We define a denotation function for abstract B terms:

$$[\![\cdot]\!]^{\mathrm{B}} \quad : \quad \mathtt{Term}\ \mathtt{Dom} \to \mathtt{Dom}$$

Denotational semantics of B

We define a denotation function for abstract B terms:

$$[\![\cdot]\!]^{\mathsf{B}} \quad : \quad \mathtt{Term}\ \mathtt{Dom} \to \mathtt{Dom}$$

where

$$\mathtt{Dom} := \sum_{x,\tau} x \in [\![\tau]\!]^{\mathsf{Z}}$$

$$\begin{cases} [\![\mathit{int}]\!]^{\mathsf{Z}} & := \mathbb{Z}^{\mathsf{Z}} \\ [\![\mathit{bool}]\!]^{\mathsf{Z}} & := \mathbb{B}^{\mathsf{Z}} \\ [\![\mathit{set}\ \alpha]\!]^{\mathsf{Z}} & := \mathcal{P}^{\mathsf{Z}}([\![\alpha]\!]^{\mathsf{Z}}) \\ [\![\alpha \times^{\mathsf{B}} \beta]\!]^{\mathsf{Z}} & := [\![\alpha]\!]^{\mathsf{Z}} \times^{\mathsf{Z}} [\![\beta]\!]^{\mathsf{Z}} \end{cases}$$

1 Introduction

2 Encoding B POs in SMT-LIB using HO

3 Results

4 Denotational semantics of B
- Definitions
- Reasoning about B terms

5 Soundness of the encoding

Denotational semantics of B

We can now state properties about B terms!

Denotational semantics of B

We can now state properties about B terms!

Theorem (Type correctness of the denotation)

Assume that $\Gamma \vdash t : \tau$*. Then,*

$$[\![t]\!]^{\mathsf{B}} = \langle T, \sigma, \mathfrak{pf}_T^{\sigma} \rangle \to \sigma = \tau$$

Denotational semantics of B

We can now state properties about B terms!

Theorem (Type correctness of the denotation)

Assume that $\Gamma \vdash t : \tau$*. Then,*

$$[\![t]\!]^{\mathsf{B}} = \langle T, \sigma, \mathfrak{pf}_T^{\sigma} \rangle \to \sigma = \tau$$

Theorem (Partial correctness of the simplifier)

Assume that $\Gamma \vdash t : \tau$*. Let* Δ *be a renaming context. Then,*

$$[\![(\!|\ t\ |\!)_{\Delta}]\!]^{\mathsf{B}} = \langle T, \tau, \mathfrak{pf}_T^{\tau} \rangle \to [\![(\!|\ \mathsf{simplifier}(t)\ |\!)_{\Delta}]\!]^{\mathsf{B}} = \langle T, \tau, \mathfrak{pf}_T^{\tau} \rangle$$

Denotational semantics of B

We can now state properties about B terms!

Theorem (Type correctness of the denotation)

Assume that $\Gamma \vdash t : \tau$*. Then,*

$$[\![t]\!]^{\mathrm{B}} = \langle T, \sigma, \mathfrak{pf}_T^{\sigma} \rangle \to \sigma = \tau$$

Theorem (Partial correctness of the simplifier)

Assume that $\Gamma \vdash t : \tau$*. Let* Δ *be a renaming context. Then,*

$$[\![(\!|\ t\ |\!)_{\Delta}]\!]^{\mathrm{B}} = \langle T,\ \tau,\ \mathfrak{pf}_T^{\tau} \rangle \to [\![(\!|\ \mathsf{simplifier}(t)\ |\!)_{\Delta}]\!]^{\mathrm{B}} = \langle T,\ \tau,\ \mathfrak{pf}_T^{\tau} \rangle$$

Remark

- Total correctness does not hold.

1 Introduction

2 Encoding B POs in SMT-LIB using HO

3 Results

4 Denotational semantics of B

5 Soundness of the encoding

What we are after

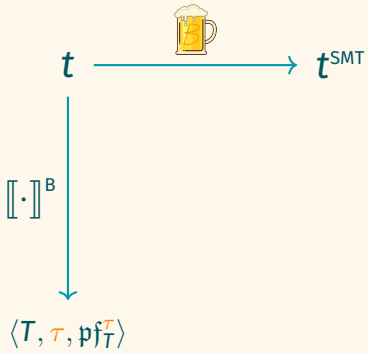
What we are after

t

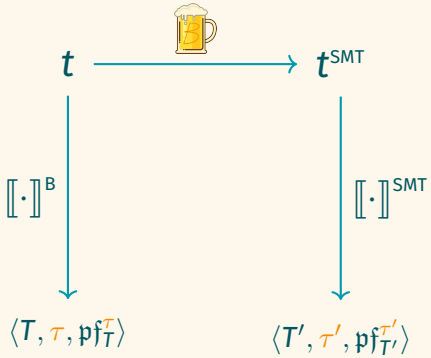
What we are after



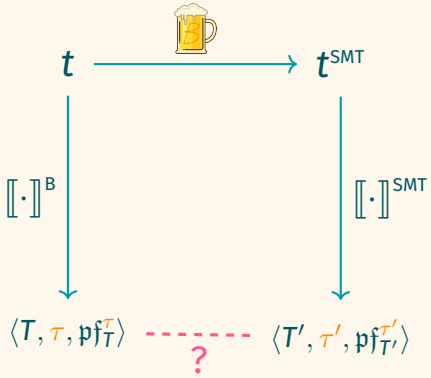
What we are after



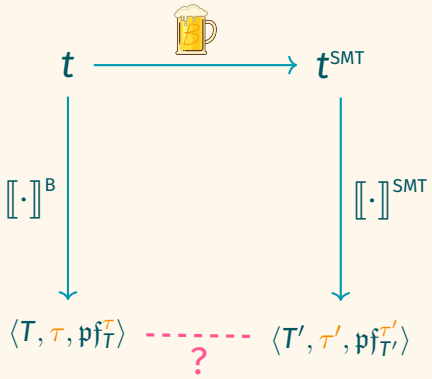
What we are after



What we are after



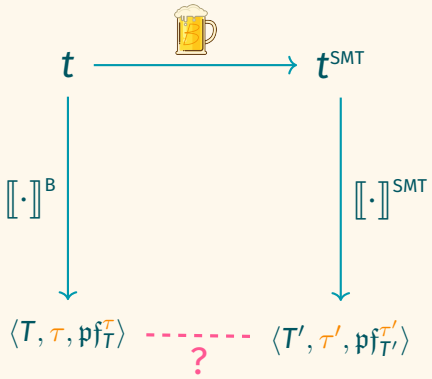
What we are after



We should at least have:

$$\tau' = \tau^{\mathsf{SMT}}$$

What we are after



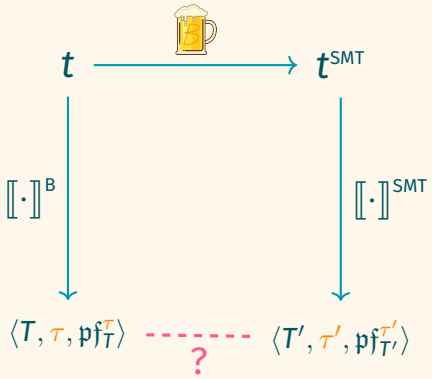
We should at least have:

$$\tau' = \tau^{\mathsf{SMT}}$$

and something like:

T “corresponds to” T'

What we are after



We should at least have:

$$\tau' = \tau^{\mathsf{SMT}}$$

and something like:

T “corresponds to” T'

which has to be formalized.

Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α:

$$[\![\alpha]\!]^{\mathsf{Z}} \cong [\![\alpha^{\mathsf{SMT}}]\!]^{\mathsf{Z}}$$

Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α:

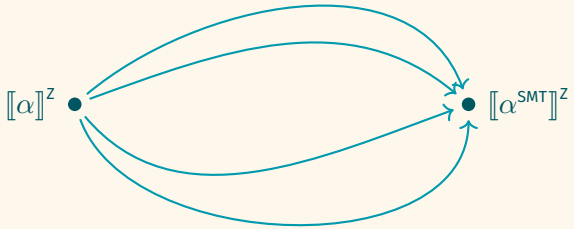
$$[\![\alpha]\!]^{\mathsf{Z}} \cong [\![\alpha^{\mathsf{SMT}}]\!]^{\mathsf{Z}}$$



Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α*:*

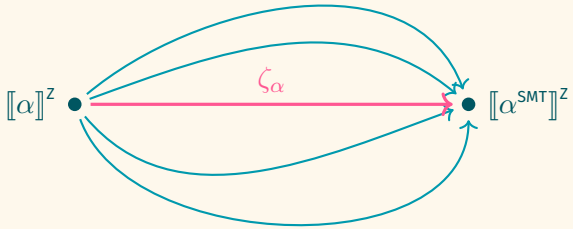
$$[\![\alpha]\!]^{\mathsf{Z}} \cong [\![\alpha^{\mathsf{SMT}}]\!]^{\mathsf{Z}}$$



Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α*:*

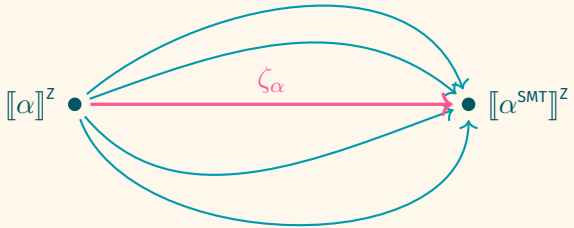
$$[\![\alpha]\!]^{\mathsf{Z}} \cong [\![\alpha^{\mathsf{SMT}}]\!]^{\mathsf{Z}}$$



Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α:

$$[\![\alpha]\!]^{\mathsf{Z}} \cong [\![\alpha^{\mathsf{SMT}}]\!]^{\mathsf{Z}}$$



Inductively-defined indexed family of canonical isomorphisms $(\zeta_\alpha)_{\alpha : \mathsf{BType}^\bullet}$

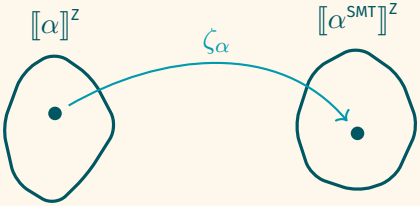
Why care about canonical isomorphisms?



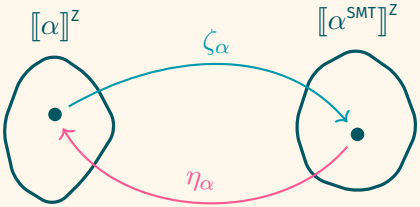
Why care about canonical isomorphisms?



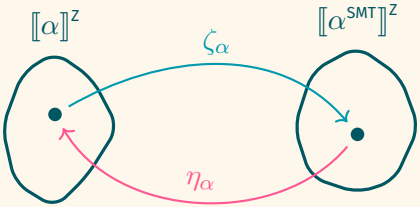
Why care about canonical isomorphisms?



Why care about canonical isomorphisms?

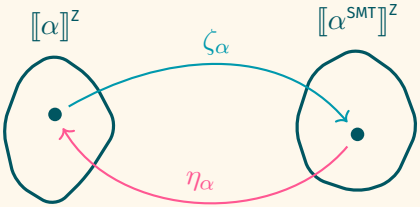


Why care about canonical isomorphisms?



Inductively-defined indexed family of retractions $(\eta_\alpha)_\alpha$: BType

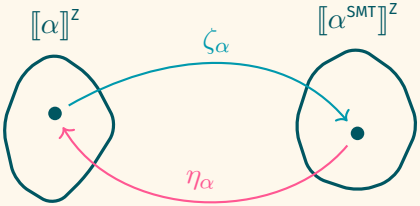
Why care about canonical isomorphisms?

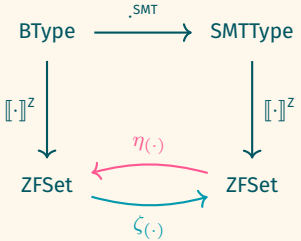


Inductively-defined indexed family of retractions $(\eta_\alpha)_\alpha$: BType

$$\eta_\alpha \circ \zeta_\alpha = \mathbb{1}_{[\![\alpha]\!]^{\mathsf{Z}}}$$

Why care about canonical isomorphisms?



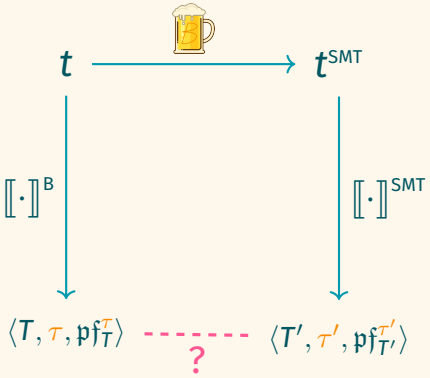


Inductively-defined indexed family of retractions $(\eta_\alpha)_\alpha$: BType

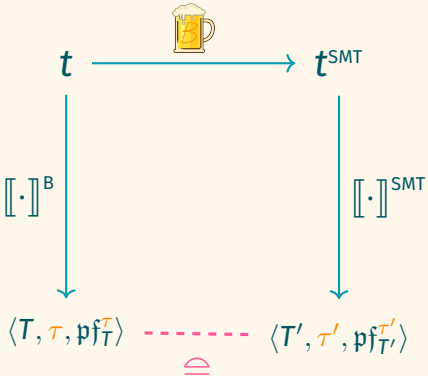
$$\eta_\alpha \circ \zeta_\alpha = \mathbb{1}_{[\![\alpha]\!]^{\mathsf{Z}}}$$

Yes, η_α is ζ_α^{-1} but we define it constructively as well.

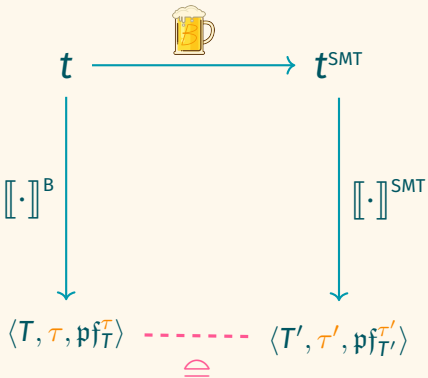
~~What we are after~~ Found it!



~~What we are after~~ Found it!



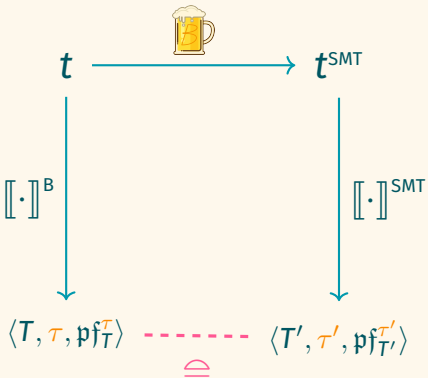
~~What we are after~~ Found it!



We define $\langle T, \tau, \mathfrak{pf}_T^{\tau} \rangle \trianglelefteq \langle T', \tau', \mathfrak{pf}_{T'}^{\tau'} \rangle$ as:

$$\tau' = \tau^{\mathsf{SMT}} \wedge \eta_\tau(T') = T$$

~~What we are after~~ Found it!



We define $\langle T, \tau, \mathfrak{pf}_T^\tau \rangle \trianglelefteq \langle T', \tau', \mathfrak{pf}_{T'}^{\tau'} \rangle$ as:

$$\tau' = \tau^{\mathsf{SMT}} \wedge \eta_\tau(T') = T$$

Now prove that this is preserved!

Conclusion

Contributions:

- **Higher-order encoding** leveraging SMT-LIB's latest features
- **Formal semantics** for subsets of B proof obligations and SMT-LIB
- **Practical framework** for set-theoretic development in Lean

Current/future work:

- **Correctness** of the encoding



VTrelat/BEer
VTrelat/ZFLean